

# Approach Summary

## Marker choice

I chose Marker1 from the supplied assets. It has three strong properties that make it practical for a lightweight custom detector:

1. The outer border is a single square ring that can be isolated as a dominant dark component.
2. The small filled square near one corner gives a stable orientation cue.
3. The inner area remains mostly open, which matches the assignment constraint that the marker leave room for encoded information.

## Detection pipeline

The implementation uses a shared pure-TypeScript image pipeline instead of a native OpenCV dependency. That keeps the core logic portable between:

- offline fixture validation in Node
- the React Native app at runtime

### Step 1: frame capture

The app requests camera permission, renders a live preview, and selects the closest supported capture size whose dimensions are as close as possible to the required 2000px to 3000px band.

### Step 2: working-image reduction

Each captured JPEG is decoded and downsampled to a working size capped at 640px on the longest side. This keeps detection latency reasonable while preserving enough shape detail for the border and anchor square.

### Step 3: dark-pixel component search

The working image is thresholded into a dark-pixel mask. Connected-component analysis is then used to identify large black structures that could be the marker border.

### Step 4: square candidate fitting

For each large component, the algorithm computes a minimum-area rectangle. Candidates that are too small or too far from square are discarded early.

### Step 5: perspective warp

Each surviving candidate is warped into a square preview image. This normalizes arbitrary in-plane rotation and gives a consistent coordinate system for validation.

### Step 6: structural verification

The warped preview is scored using three checks:

1. Border continuity: every edge band must remain strongly dark.
2. Anchor square presence: exactly one corner window must contain a compact filled square with the expected size profile.
3. Inner content presence: the center region must contain non-white signal so empty or near-empty lookalikes are rejected.

If all checks pass, the best-scoring candidate is treated as the detected marker.

## **Step 7: orientation correction and extraction**

The winning quadrilateral is scaled back to the original full-resolution image and warped again to the final 300x300 output size. The image is then rotated in 90° steps until the orientation anchor lands in the top-left corner.

## **Why this approach**

- It satisfies the assignment requirement to detect only the intended marker by combining geometry and content checks.
- It keeps the extraction tightly cropped and normalized to a fixed square result.
- It avoids heavy native computer-vision dependencies, which makes the repo easier to review and set up.

## **Known tradeoffs**

- The current scan loop uses sequential image capture rather than a native frame processor, so it is simpler than a fully optimized production pipeline.
- The detector is strongest for high-contrast scenes and near-frontal views. A native frame processor plus lower-level camera access would be the next step for pushing scan latency lower and making perspective handling more aggressive.